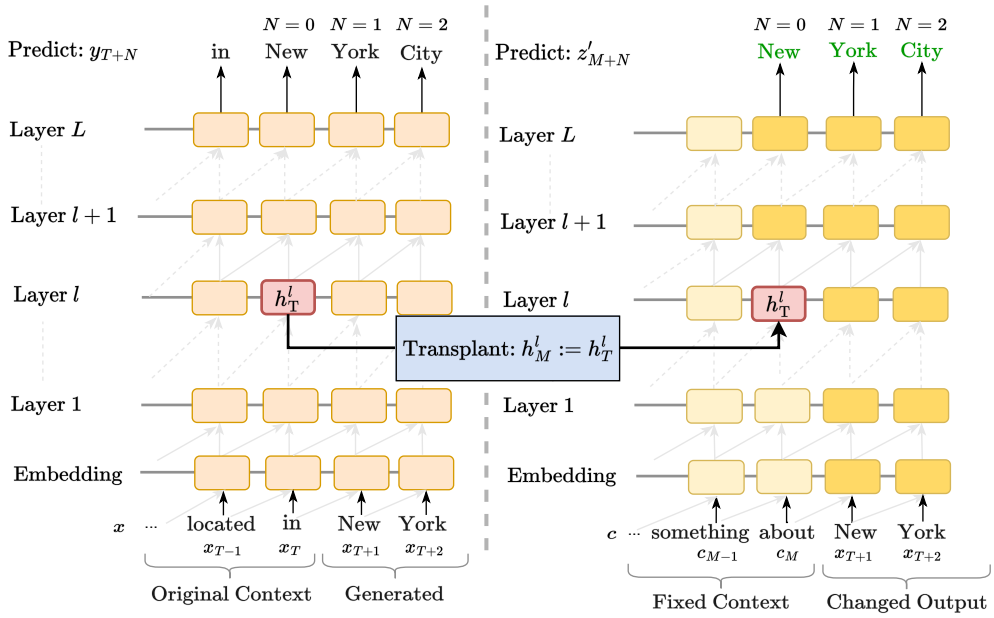

Figure 2: Illustration of Fixed prompt Causal Intervention. The left and right sides represent two different transformer model runs. On the left hand side, we have the original run of *Madison Square Garden ... in New York*. We transplant the hidden state, h_T^l to the other transformer model run, which has a fixed generic context, *Tell me something about*, as its input. With h_T^l replacing the hidden state at h_M^l, we measure the tendency of this modified transformer run to reveal the probability distribution in h_T^l. In such cases, it would reveal that h_T^l was predicting, for instance, 'New York City.'

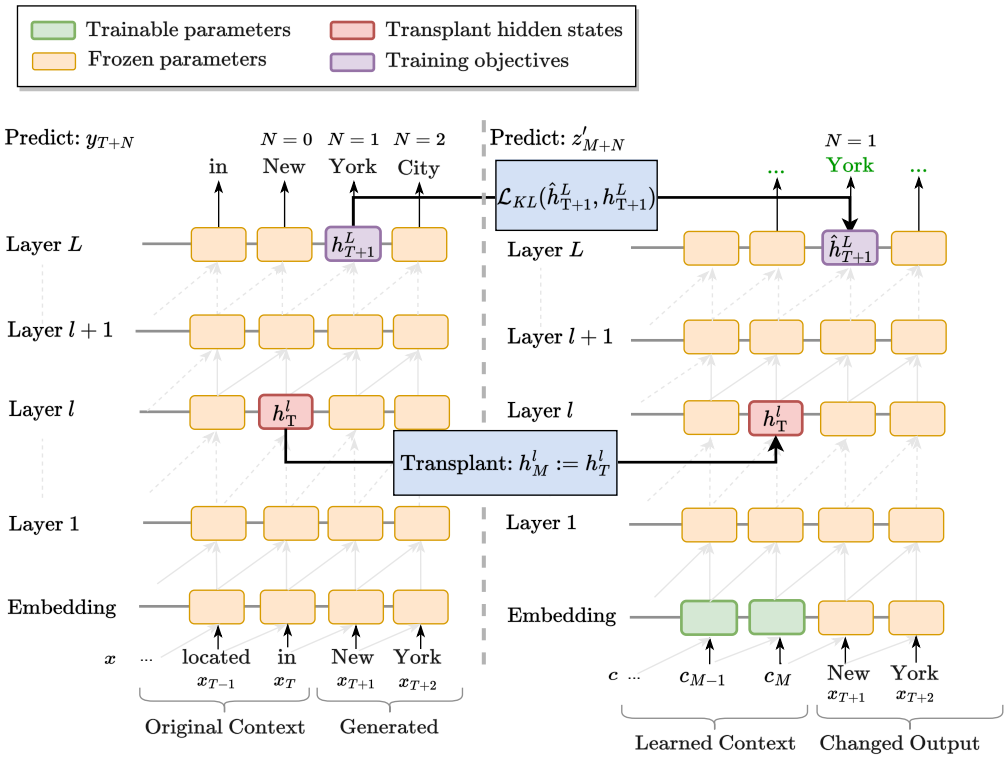

Figure 3: Learned context prompt Causal Intervention Overview. The left and right sides represent two different transformer model runs. The general setup is the same as Figure 2. The difference lies in the context provided in the transformer run on the right hand side. Instead of manually thinking of a context, we provide a learned context to increase the tendency of decoding the subsequent tokens predicted by h_T^l. We do so by training the context, c, with L_{KL} criterion and the objective to match the subsequent token prediction, such as 'York' in this instance.

the hidden state. For each decoder layer l, we train the corresponding prefix $c_{\text{opt}}^{(l)} = [c_1^{(l)}, ..., c_M^{(l)}]$ to maximize the probability of the model yielding the exact subsequent phrase after the original context. In particular, we conduct the same causal intervention in the hidden states h_T^l. We then optimize the probability distribution of the subsequent generation under the learned context to be the same as the original model when all its previous generation is given correctly:

$$\text{argmin}\, \text{KL}(\hat{y}_{M+N}\, ;\, y_{T+N}) \tag{10}$$

Where the predicted distribution $\hat{y}_n$ is given using the same intervention as described in Eq. 9:

$$\hat{y}_{M+n} = G([c_1, ..., c_M, x_{T+1}, .., x_{T+N}] \\ ||\, h_M^l := h_T^l) \tag{11}$$

We hence optimize this objective with the model frozen and only prefix left to be trained. Notably, our approach is different from the implementation of prefix tuning (Li and Liang, 2021) in the sense that we back-propagate the gradient through the model instead of a temporary MLP, as empirically it produces a significantly better optimized context.

3 Experiments and Results

3.1 Data

We perform evaluation on samples of the Pile (Gao et al., 2020), which is the 825GB dataset used to train GPT-J-6B (Wang and Komatsuzaki, 2021) as well as other LLMs.

To train the linear models, we sample 100,000 tokens that have an average of 518 sized-context. Amongst the 100,000 token samples, we use 10,000 of them to train for our learned prompt experiment. For testing our methods, we sample another 1000 tokens that have an average previous context length of 535. To simplify our analysis of the degree to which single hidden token representations encode subsequent n-grams, we draw our samples from contexts in which the original transformer model made a correct prediction.

More specifically, we randomly sampled train and test data points from the subset of token locations where the autoregressive transformer under consideration correctly predicts the following token. In Table 1, we break down the types of tokens present in the testing data by categorizing the last token (T) of the prefix as well as the generated tokens outputs of GPT-J , through greedy (argmax) decoding, at $N = 0, 1, 2, 3$ with respect to various properties, such as whether they are lower-cased tokens that start with a space, or are numerical tokens, and so on.

3.2 Evaluation Metrics

For evaluation we adopt the same metrics used in prior related work Din et al. (2023), namely Precision@k and Surprisal.

Precision@k measures the appearance of the top probability token in the output at N tokens ahead we predict from the hidden state with respect to the observed top-k tokens from GPT-J-6B model output. Higher values are better here because these mean the actual token at the corresponding future token was accurately predicted.

Surprisal, on the other hand, is the minus log probability according to the GPT-J-6B model output of the highest probability token according to the proposed probing methods. Lower is better for this measure because such values imply that the top predicted tokens are deemed probable by the model.

3.3 Experimental Setup

Linear Model We train two types of linear models — one with an output space of 4096 (the hidden representation size used by GPT-J-6B), and the other one with 50,400 (the vocabulary space of the same). GPT-J-6B comprises 28 layers. We train 4 instances for each of these layers, one for each different "future" token position we consider ($n = 0, 1, 2, 3$). As input we accept the source hidden state, i.e., h_T^l. Our output is either the hidden state, i.e., h_{T+N}^L or the decoded output at the position (vocabulary distribution) $T + N$.

Fixed Prompt Causal Intervention This is an evaluation-only setup where we choose four generic context prompts and perform causal intervention on these contexts as shown in Figure 2. The four fixed context prompts that we test are:

- `Hello! Could you please tell me more about "`
- `The multi-tokens present here are "`
- `The concepts in this hidden state listed are: (`
- `<|endoftext|> This state is describing about the following concept:`

The hidden states are gathered from layer l of the last token of the context tokens and are transplanted into the hidden representation of the last token in the generic prompts at the same layer l.

Properties	Last Original Context Token	N = 0	N = 1	N = 2	N = 3	Examples
Lowercase No Space	12	14.5	18.1	13.1	13.4	‘itability’, ‘aka’, ‘ension’
Lowercase With Space	42	39.1	37.1	38.4	36.7	‘ sense’, ‘ tests’, ‘ punitive’
Uppercase No Space	2.4	2.7	2.2	2.8	1.6	‘V’, ’TABLE’, ’SE’
Uppercase With Space	1.9	2.4	1.1	1.5	1.7	‘ STAR’, ‘ UK’, ‘ USA’
Token length < 4	57.8	59.8	64.3	59.9	63.2	‘*’, ‘ate’, ‘</’
Token length ≥ 4	42.2	40.2	35.9	40.5	37	‘ validation’, ‘ Subaru’, ‘ulsion’
Punctuation	15.7	14.5	17.3	15.2	19	‘-’, ‘.’, ‘</’
Numerical	2.4	2.7	1.9	3.2	2.8	‘1998’, ‘001’, ‘5’

Table 1: Data Frequency of different token properties on the Last Prefix Tokens and GPT outputs at N=0,1,2,3. Each number in the table is a percentage of the test dataset, which is of size 1000.

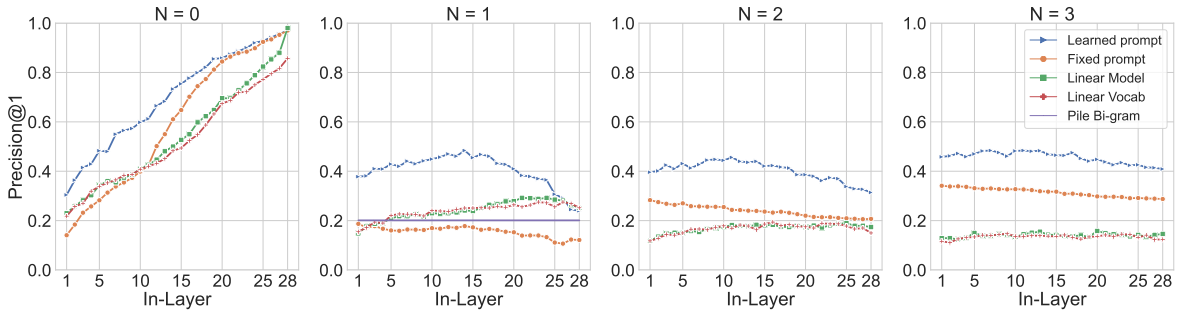

Figure 4: Accuracy (Precision@1) using the transplanted hidden representation. The $N = 0$ case models immediate next-token prediction, and $N \geq 1$ are the subsequent-token cases that are the focus of our work. The learned prompt is best able to recover future token information from hidden states of a preceding individual token, with predictive accuracy peaking at middle layers, with more than double the accuracy of a bigram baseline. A linear model predicting the hidden state fares comparably to predicting directly into the output vocabulary.

Learned Prompt Causal Intervention We then compare with trained prompts with the same token length as the fixed prompts. We train a soft prompt for each layer l from 1 to 28. Each learned prompt is trained by maximizing the probability of generating the token from the prefix context at the penultimate layer, when the hidden state is transplanted at layer l at the last token of the soft prompt, in the same way as the fixed prompts are applied. We train a prefix with a length of 10. This method performs best and is our main method.

3.4 Unveiling Subsequent Tokens

Figure 4 and Figure 5 illustrate the difference between our method and the baselines. The learned prompt optimized with the objective of predicting the next token (N=1) has the best performance. On average, the precision@1 is 24.8% higher, precision@5 is 25.3% higher, and precision@10 is 25.1% higher than the **best** baseline method. The bigram baseline[1] at N=1 is shown as a horizontal line; the bigram model i achieves 20.1% accuracy. For surprisal, the learned prompt also has the lowest value, which indicates its efficacy at maximally unveiling the information behind the hidden states.

[1]The bigram baseline is collected from 900,000 documents from the Pile dataset.

3.5 Contexts of Accurate Predictions

To further explore the contexts in which these methods seem better (or worse) able to predict subsequent tokens, we categorize input token (the last original context token) into eight (non-mutually exclusive) categories, shown in Table 3. We report the model accuracies when using layer 14, where the learned prompt model peaks.

While all categories of token types are predicted better by the learned prompt than by the linear model, the relative improvement is highest when the last context token is a lowercase token preceded by a space, or a longer token. This suggests that information about how to complete long words may not be immediately accessible by a linear model decoder, but that they can be made accessible by using the parameters of the pretrained model as done by the learned prompt intervention method.